

Rapport d'analyse et de conception — DataValueXploring

Projet : réduire durablement les accidents de la route par la rénovation d'infrastructure **Rôle incarné** : agence de voirie / DOT d'une collectivité territoriale **Données** : *US Accidents* (Kaggle, Sobhan Moosavi) — Feb 2016 → Mar 2023, 7 728 394 accidents, 46 colonnes, 49 États **Livrables techniques** : rendu/ (notebook exécutable de bout en bout, README, requirements figés, tests pytest, Dockerfile)

1. Le rôle et l'enjeu métier

Le sujet impose d'incarner un acteur réel et d'en tirer une solution *créatrice de valeur, compréhensible et exploitable par un décideur*. Nous avons choisi la **collectivité territoriale**, et plus précisément son **gestionnaire d'infrastructure routière (DOT)**, pour une raison simple : c'est le rôle qui rend les données *actionnables*.

Un DOT dispose d'un **budget de travaux fini** et doit décider **où** intervenir et **quoi** installer. Aujourd'hui cette priorisation est surtout **réactive** : remontées terrain, accidents médiatisés, arbitrages politiques. Notre fil rouge :

« Comment réduire durablement les accidents sur notre réseau, avec un budget de travaux limité ? »

Ce fil rouge se décline en trois questions, chacune avec sa **nature** propre — c'est la colonne vertébrale de tout le projet :

Question métier	Nature	Méthode retenue
OÙ rénover en priorité ?	concentration → descriptif	agrégation géospatiale H3 + exposition OSM
QUOI installer sur place ?	attribution → ML supervisé (moteur de preuve)	modèle de gravité → effets ajustés → prescription
QUAND agir temporairement ?	conjoncturel → descriptif	patterns temporels / météo (visualisation)

Le périmètre d'action du rôle délimite aussi ce qu'on ne fait **pas** : on agit sur l'infrastructure (feux, passages piétons, stops, modération de trafic), pas sur la météo ni sur le comportement individuel des conducteurs.

2. Les données : ce qu'elles permettent, ce qu'elles interdisent

L'analyse préliminaire (EDA PySpark sur les 7,73 M lignes) a posé très tôt les contraintes structurantes :

- Couverture très déséquilibrée** : Top 3 États = 41 % du dataset (CA 1,74 M, FL 880 k, TX 583 k). La collecte via API trafic (MapQuest/Bing) crée un **biais de reporting** : peu de lignes ≠ peu d'accidents. Conséquence méthodologique : on ne compare jamais les États entre eux ; on raisonne à *l'intérieur* d'un territoire.
- Sévérité très déséquilibrée** : classe 2 = 79,7 %, classe 4 = 2,6 %. Conséquence : binarisation (`grave` = `sévérité` ≥ 3 , soit 19,5 %) et métriques adaptées.
- Valeurs manquantes lourdes** : `end_lat/end_lng` 44 % (emprise géographique abandonnée), `precipitation_in` 28 %, `wind_chill_f` 26 % (supprimée, redondante avec la température).

Et dès cette phase, les **deuils** ont été actés — ils n'ont jamais changé depuis :

Renoncement	Raison
Causalité d'un aménagement	pas de panel avant/après installation → associations uniquement
Profils conducteurs (âge, alcool, vitesse)	absents des données
Coût réel des accidents	<code>severity</code> est le seul proxy
Couverture homogène	biais de reporting

Emprise géographique	44 % de end_lat/lng manquants
Description textuelle	NLP hors scope

3. Le cheminement : itérations, impasses et pivots

Cette section raconte honnêtement **comment** nous sommes arrivés à la solution — y compris ce qui n'a pas marché. C'est là que se sont joués les principaux apprentissages.

Itération 0 — EDA et premières idées (début mai)

Premier notebook (01_eda) : volumétrie, valeurs manquantes, distribution de la sévérité, couverture géographique et temporelle, et un premier signal fort : la **sévérité moyenne avec/sans chaque élément d'infrastructure**. Résultat brut : `junction` était la *seule* feature associée à une hausse de sévérité (+0,092), tous les équipements semblaient protecteurs (`crossing` -0,167, `traffic_signal` -0,144...).

À ce stade, plusieurs idées étaient sur la table, toutes ne survivront pas :

- un modèle prédictif de **durée de blocage** (`end_time` - `start_time`) → abandonné (44 % de NA sur les coordonnées de fin rendaient les données de fin douteuses) ;
- une hésitation de périmètre : Californie entière, Los Angeles County, ou un État médian (Caroline du Sud) → tranchée plus tard par la **généricité** (entraînement US entier, la Californie comme simple terrain de démonstration paramétré par une variable `STATE`) ;
- un projet initial en **Spark MLlib** de bout en bout → remplacé ensuite par scikit-learn sur échantillon maîtrisé (voir §5), MLlib restant documenté comme alternative scalable.

Itération 1 — Points noirs géospatiaux (02_points_noirs)

Première brique « OÙ » : binning **H3 résolution 8** (~0,7 km², l'échelle d'un carrefour), score de zone `charge` = `n` × `sévérité moyenne` , carte Folium de la Californie, identification des carrefours sans équipement. Deux analyses de cette itération ont survécu jusqu'au rendu final :

- la **validation de stabilité temporelle** (scores 2016-2019 vs 2020-2023) — l'idée que des points noirs *structurels* doivent persister dans le temps ;
- l'analyse **jours fériés / grands événements** (Noël, Thanksgiving, Super Bowl...) — au résultat contre-intuitif (la gravité ne s'emballe pas pendant les fêtes).

Itération 2 — L'impasse : le modèle tautologique (03_modele_attribution)

C'est l'erreur la plus instructive du projet. Pour répondre à « QUOI », nous avons entraîné un **RandomForestRegressor** **prédisant la charge d'une zone à partir des comptages d'infrastructure de cette zone** (`n_junction` , `n_crossing` ...). Le R^2 était flatteur — et pour cause : **la cible était une fonction quasi arithmétique des features**. Les comptages d'accidents par élément *composent* la charge : le modèle apprenait une identité comptable, pas un phénomène. Ses « importances » ne mesuraient rien d'autre que la composition du territoire.

Leçon retenue (et gravée dans le cahier des charges du rendu) : vérifier systématiquement qu'aucune feature ne contient — même transformée — la cible. Cette leçon a produit deux garde-fous dans le livrable final : le changement d'**unité d'analyse** (l'accident, pas la zone) et des **tests anti-fuite automatisés** (assertions bloquantes dans le notebook).

De cette itération, nous avons néanmoins conservé le **moteur de recommandation** (aléa local par sur-représentation/lift, mesure manquante, effet conditionnel) — l'idée était bonne, c'est son alimentation par un modèle tautologique qui ne l'était pas.

Itération 3 — L'exposition routière (04_exposition)

Une objection restait sans réponse : « *vos zones sont-elles dangereuses, ou juste très fréquentées ?* ». Réponse : normaliser par l'**offre routière** — longueur de voirie et nombre d'intersections par cellule, récupérés via OSM (`osmnx`). Le re-classement par `accidents/km` fait remonter des **points noirs concentrés** masqués par le volume, et redescendre des zones simplement denses en routes. Cette brique a aussi apporté la typologie « point noir concentré vs réseau étendu ».

Itération 4 — Reconstruction *from scratch* du rendu (juin)

Plutôt que d'empiler les notebooks d'expérimentation, nous avons rédigé un **cahier des charges complet** (le `CLAUDE.md` du rendu) et reconstruit un **notebook unique**, exécutable de bout en bout sans intervention. Le cadrage ML y est reformulé :

*On entraîne un modèle qui prédit la gravité **binaire** d'un accident à partir de l'infrastructure **plus** le contexte (météo, heure, État — comme variables de contrôle). Le produit n'est pas la prédiction : c'est **l'explication**. La cible n'est plus une fonction de ses features : la tautologie est éliminée.*

Cette itération a apporté : la comparaison de **5 modèles** (Dummy → LogReg → arbre → RandomForest → HistGradientBoosting) à protocole identique, le **split temporel** (train < 2022, test 2022-2023), les métriques adaptées au déséquilibre (PR-AUC, rappel sur la classe grave — l'accuracy est explicitement bannie), la validation de stabilité, l'exposition OSM intégrée, la synthèse décisionnelle et l'analyse critique.

Elle a aussi eu son lot d'obstacles d'ingénierie, instructifs en eux-mêmes :

- **L'API Overpass (OSM) s'est révélée le maillon fragile** : requêtes qui bloquent sans timeout, throttling agressif, et un piège subtil — la clé de cache d' `osmnx` dépend du timeout configuré, si bien qu'un cache « chauffé » avec un réglage différent était invisible. Résolution : cache local livré avec le dépôt, timeout par requête, **budget temps global** et **dégradation propre** (sans OSM, le classement par volume est conservé et la limite documentée). Le « Run All » se termine *toujours*.
- **La suite de tests pytest a prouvé sa valeur immédiatement** : à sa première exécution, elle a détecté qu'une édition antérieure avait **amputé la partie « Analyse critique » du notebook** — une régression de contenu invisible à l'œil nu, restaurée aussitôt. C'est l'argument vécu en faveur des protocoles de tests exigés par la grille.

Itération 5 — Le pivot final : des explications *vraiment* claires (début juillet)

En préparant la soutenance, un malaise est apparu : la chaîne « modèle → SHAP → prescription » n'était **pas honnête avec elle-même**. Deux problèmes précis :

1. le **SHAP global** donnait des lectures piégées par les confondants (les feux « aggravants » parce qu'ils *marquent* les carrefours denses) ;
2. la prescription finale reposait en réalité sur des **moyennes brutes conditionnelles** — alors que l'argument de vente du modèle était justement de *contrôler les confondants*.

Le pivot : remplacer SHAP comme preuve principale par deux outils faits exactement pour notre question (« *quel est l'effet net de chaque équipement, à contexte égal ?* ») :

- les **odds ratios de la régression logistique** (avec IC 95 %) — le standard de l'analyse de facteurs de risque, lisible par un non-spécialiste ;
- la **g-computation** sur le modèle retenu — on prédit la probabilité de gravité pour les mêmes accidents avec l'équipement « éteint » puis « allumé », toutes choses égales par ailleurs ; le résultat est en **points de probabilité**, traduisible en **accidents graves évités**.

Résultat : les deux lectures **concordent (10/12 équipements)** et renversent la lecture naïve — à contexte égal, passage piéton (OR 0,41 ; -20 pts), stop (0,27 ; -19 pts) et feux (0,39 ; -15 pts) sont **protecteurs** ; carrefour (1,2) et voie ferrée (1,7) **aggravants**. La prescription par zone est désormais pilotée par l'**effet ajusté conditionnel du modèle** (Δ_{adj} (mesure | aléa local)), contre-vérifié par l'effet brut, et l'impact est exprimé en **accidents graves potentiellement évités**. SHAP est conservé en **annexe de robustesse** — avec l'explication pédagogique de pourquoi une requête d'*attribution* (SHAP) et une requête d'*intervention* (g-computation) peuvent différer.

4. La solution finale

```
Setup reproductible → Chargement Spark (schéma explicite) → Nettoyage explicite & testé
├─ A. EDA : volumétrie, biais, sévérité, « QUAND » (météo, heures, fêtes)
├─ B. OÙ : H3 rés. 8 → charge pondérée gravité → exposition OSM (acc/km)
│     + validation de stabilité temporelle (p = 0,776)
├─ C. QUOI: 5 modèles comparés (split temporel, PR-AUC vs plancher)
│     → odds ratios (IC 95 %) + g-computation = effets ajustés
```

	→ SHAP en contrôle de robustesse
	D. Synthèse : par zone, aléa local × mesure manquante × effet ajusté
	conditionnel → recommandation + impact (graves évités) → CSV + carte
	E. Analyse critique : ce qui marche / pas, limites, périmètre, décision terrain

Résultats clés (exécution réelle, embarquée dans le notebook) :

Résultat	Valeur
Concentration du risque (Californie)	50 zones (0,08 % de 58 857) = 5,3 % de la charge (×62)
Stabilité temporelle des points noirs	Spearman ρ = 0,776 ; les 50 zones restent top 5 %
Modèle retenu (HistGradientBoosting)	PR-AUC 0,125 = ×2 vs plancher (0,064), ROC-AUC 0,68
Effets ajustés (OR / g-computation)	concordance 10/12 ; équipements régulateurs protecteurs
Exposition OSM	42/50 zones (accidents/km + intersections)
Impact estimé cumulé (top 50, ex-ante)	≈ 6 860 accidents graves potentiellement évités
Recommandations	6 types différents ; 13 zones « facteur humain » signalées

5. Justification des choix méthodologiques

Choix	Alternatives écartées	Justification
PySpark pour le traitement	pandas pur	7,73 M lignes : agrégations et split US-wide en Spark ; bascule pandas uniquement une fois le volume réduit
sklearn sur échantillon 800 k pour la comparaison	Spark MLlib bout-en-bout	protocole de comparaison riche (5 modèles, PR-AUC, SHAP) impossible en MLlib pur dans le temps imparti ; échantillonnage stratifié dans le temps, plafonné par une constante documentée ; MLlib cité comme voie de scalabilité
Cible binaire grave = sévérité ≥ 3	multiclasse 1-4	classe 2 = 80 % → métriques multiclassées illisibles ; message décisionnel clair
Split temporel train < 2022 / test 2022-23	split aléatoire	simule l'usage réel (prédire l'avenir) et évite la fuite temporelle
class_weight='balanced'	SMOTE	ne fabrique pas de données synthétiques
PR-AUC + rappel grave , accuracy bannie	accuracy	à 80/20, prédire « non-grave » partout donne 80 % sans rien apprendre
H3 résolution 8	grille carrée ; rés. 7/9	cellules équi-surface (comptage = densité comparable) ; ~0,7 km² = échelle d'un carrefour
Odds ratios + g-computation comme preuve	SHAP seul	répondent littéralement à la question métier « effet net à contexte égal » ; lisibles ; concordance = robustesse ; SHAP global piégé par les confondants
Effet conditionnel pour prescrire	effet moyen global	un effet moyen recommanderait le même aménagement partout (artefact) ; le conditionnement produit des recommandations variées et locales
Généricité : entraînement US, STATE paramétrable	spécialisation Californie	le code ne contient rien de spécifique à la CA ; changer une constante rejoue l'analyse ailleurs

6. La valeur créée pour le décideur

On transforme un budget limité en liste d'interventions priorisée, justifiée par la donnée et explicable.

Concrètement, le DOT ouvre `zones_prioritaires.csv` (50 zones triées par impact estimé) et la carte interactive : chaque ligne est une **décision de travaux argumentée** — où (cellule H3, ~un carrefour), quoi (l'aménagement qui manque et qui est efficace *dans ce contexte*), et l'enjeu (accidents graves potentiellement évités). Les analyses « QUAND » (le brouillard n'aggrave pas, les fêtes non plus) évitent par ailleurs de dépenser du budget travaux là où des **mesures dynamiques** suffisent.

La priorisation est **défendable** : concentration mesurée, stabilité temporelle validée, exposition contrôlée, effets ajustés concordants entre deux méthodes indépendantes — le DOT peut justifier ses choix d'investissement face à des élus ou des riverains, indépendamment des accidents médiatisés.

7. Limites et périmètre d'usage

- **Associations, pas causalité** : aucun panel avant/après installation. L'« impact estimé » est **ex-ante** ; il sert à *classer* les interventions, pas à garantir un résultat.
 - **Biais de reporting** : la couverture des données est inégale ; le modèle est fiable sur les territoires à fort volume, pas sur les zones peu instrumentées.
 - **Pouvoir prédictif modéré assumé** (PR-AUC 0,125, x2 vs plancher) : la gravité dépend de facteurs absents des données (vitesse réelle, alcool). Le modèle est un *révélateur de leviers*, pas un oracle — conforme au sujet (« la performance brute n'est pas un objectif en soi »).
 - **Exposition OSM partielle** (42/50 zones) : l'API Overpass limite les requêtes ; le cache local livré permet le rejeu hors-ligne, le reste est classé par volume (documenté).
 - **Mesure d'impact réel** : nécessiterait un pilote terrain (5-10 zones) évalué en **différence de différences** (zones traitées vs témoins comparables), avec pour KPI le coût par accident grave évité. C'est notre première perspective.
-

8. Reproductibilité et validation

- **Run All sans intervention** : données auto-téléchargées (kagglehub, repli CSV local), graines fixées, constantes nommées en tête, versions figées (`requirements.txt`).
 - **Tests à deux niveaux** : assertions bloquantes dans le notebook (intégrité du nettoyage, anti-fuite du pipeline ML) + suite **pytest** sur le livrable (validité du notebook exécuté, invariants métier du CSV, artefacts présents).
 - **Docker** : image fournie (Python 3.12 + JDK encapsulés), build vérifié.
 - **Généricité** : variable `STATE` unique ; rien de codé en dur pour la Californie.
-

9. Perspectives

1. **Causalité** : panel avant/après si les dates d'installation deviennent disponibles ; à défaut, différence de différences sur un pilote.
2. **Exposition complète** : trafic AADT + OSM sur toutes les cellules → vrais taux d'accident.
3. **Risque conjoncturel** : modèle dédié « QUAND » pour piloter des panneaux à messages variables.
4. **Industrialisation** : extraction en package `src/` + CI, au-delà du notebook unique.